

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA1205	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	30%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	Shafina.A	Reg.No.:	192521474
Department:	B TECH IT	Date:	

ANNEXURE A

Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.

DMA-Based Data Transfer

ALGORITHM DMA_Transfer

BEGIN

Initialize DMA_Controller

Set Source = IO_Device

Set Destination = Main_Memory

Set Transfer_Size

Configure DMA_Controller

Enable DMA_Interrupt

Start DMA_Transfer

```
WHILE Transfer_Not_Complete DO
```

```
    CPU performs other tasks
```

```
END WHILE
```

```
WHEN DMA_Interrupt occurs DO
```

```
    IF Transfer_Status = SUCCESS THEN
```

```
        Mark transfer as Complete
```

```
    ELSE
```

```
        Handle Transfer_Error
```

```
    END IF
```

```
    Clear Interrupt
```

```
END WHEN
```

```
Return Control to CPU
```

```
END
```

Purpose: DMA transfers data directly between the I/O device and memory, minimizing CPU involvement.

2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.

Priority-Based Vectored Interrupt Handling

```
ALGORITHM Interrupt_Handler
```

```
BEGIN
```

```
    Initialize Interrupt_Controller
```

```
    WHILE System_Is_Running DO
```

```
        IF Interrupt_Received THEN
```

```
            Identify all Active Interrupts
```

```
            Select Highest_Priority Interrupt
```

```
            Get Interrupt_Vector
```

```

        Save CPU_Context

        Jump to Service_Routine(Interrupt_Vector)

        Execute Service_Routine

        Clear Interrupt

        Restore CPU_Context

    END IF

END WHILE

```

END

Purpose: The highest-priority interrupt is serviced first, reducing response time for critical devices.

3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.

Dynamic I/O Mapping Selection

ALGORITHM Select_IO_Method(Latency, Bandwidth)

BEGIN

```

    IF Latency <= LOW_LATENCY_LIMIT AND
       Bandwidth >= HIGH_BANDWIDTH_LIMIT THEN

```

```

        Method ← MEMORY_MAPPED_IO

```

```

    ELSE

```

```

        Method ← IO_MAPPED_IO

```

```

    END IF

```

```

    RETURN Method

```

END

Purpose: Selects an appropriate I/O mechanism according to performance requirements.

4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.

PCIe, USB and Thunderbolt Bus Arbitration

ALGORITHM Bus_Arbitration

BEGIN

 Initialize PCIe, USB, Thunderbolt

 WHILE Requests_Exist DO

 Collect requests from all devices

 FOR each Request DO

 Determine Bus_Type

 Determine Priority

 Determine Bandwidth_Requirement

 END FOR

 Select Highest_Priority Request

 IF Bus_Type = PCIe THEN

 Allocate PCIe resources

 ELSE IF Bus_Type = USB THEN

 Allocate USB bandwidth

 ELSE IF Bus_Type = Thunderbolt THEN

 Allocate Thunderbolt resources

 END IF

 Grant Bus_Access

 Transfer Data

 Release Bus

 END WHILE

END

Purpose: Efficiently allocates communication resources among different I/O buses.

5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Hybrid I/O Communication

ALGORITHM Hybrid_IO

BEGIN

FOR each IO_Device DO

Determine Device_Speed

IF Device_Speed = LOW THEN

Start Polling

WHILE Data_Available DO

Read Data

Process Data

END WHILE

ELSE

Configure DMA

Enable Interrupt

Start DMA_Transfer

CPU performs other tasks

WHEN DMA_Interrupt occurs DO

Check Transfer_Status

IF SUCCESS THEN

Process Received Data

ELSE

Handle Error

END IF

Clear Interrupt

END WHEN

END IF

END FOR

END

Purpose: Polling is used for simple low-speed devices, while **DMA + interrupts** is used for high-speed devices to achieve high throughput and reduce CPU overhead.

Code of Conduct:

I _____YUVASRI.S (192521032)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
----------	-----------	---------------------	----------------------	----------------------	---------------------

Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient